

Fonctions standards, opérateurs et clauses optionnelles SQL

Partie LDD

Requête SQL		Exemple d'utilisation										
CREATE TABLE	CREATE TABLE nom_table (Nom_Colonne1 type_colonne1 [[CONSTRAINT] contrainte-col]* ... [CONSTRAINT] contrainte-table);	Exemple : CREATE TABLE Vehicule(immat_Vehicule VARCHAR(10) primary key, marque VARCHAR(20) NOT NULL CHECK(marque in('Peugeot','Renault','Fiat','Opel','Volkswagen','Citroën')), model VARCHAR(20) NOT NULL, date_acq YEAR NOT NULL); Exemple: CREATE TABLE Location(immat_Vehicule VARCHAR(10) REFERENCES vehicule(immat_Vehicule)ON DELETE CASCADE, NCIN_Cli VARCHAR(8) REFERENCES Client(NCIN_Cli)ON DELETE CASCADE, date_loc TIMESTAMP DEFAULT CURRENT_TIMESTAMP C HECK(date_loc<=NOW()), duree_loc INT(3) CHECK(duree_loc>0), cout_loc DECIMAL(8,3) CHECK(cout_loc>0), CONSTRAINT PK_Location PRIMARY KEY(immat_Vehicule,NCIN_Cli)); Pour attribuer la date d'aujourd'hui comme valeur par défaut, il faut utiliser le type "timestamp" sinon ça ne marche pas avec le type date <table><tr><td>DATE</td><td>Date (ex: 2024-11-04)</td></tr><tr><td>TIME</td><td>Heure (ex: 23:44:05)</td></tr><tr><td>DATETIME</td><td>Date et heure (ex: 2024-11-04 23:44:05)</td></tr><tr><td>YEAR</td><td>Année (ex: 2024)</td></tr><tr><td></td><td></td></tr></table>	DATE	Date (ex: 2024-11-04)	TIME	Heure (ex: 23:44:05)	DATETIME	Date et heure (ex: 2024-11-04 23:44:05)	YEAR	Année (ex: 2024)		
	DATE	Date (ex: 2024-11-04)										
TIME	Heure (ex: 23:44:05)											
DATETIME	Date et heure (ex: 2024-11-04 23:44:05)											
YEAR	Année (ex: 2024)											
Alter Table	Ajout de colonnes à une table	Ajouter à la table Client un champ nommé Tel_Cli de type texte et de taille égale à 10 ALTER TABLE Client ADD COLUMN (Tel_Cli) varchar(10); <i>N.B : l'utilisation de () après le mot clé "COLUMN" est facultatif quand il s'agit de l'ajout d'une seule colonne</i>										
	Modification d'une colonne de la table	Modifier le type de donnée de la colonne " Tel_Ci " de la table " Client " en numérique de 8 chiffres. ALTER TABLE Client MODIFY COLUMN Tel_Cli INT(8);										
	Suppression des colonnes d'une table	Supprimer la colonne " Adresse_Cli " dans la table " Client ". ALTER TABLE Client DROP COLUMN Adresse_Cli;										

	Ajout d'une contrainte	Vérifier que les valeurs de "Annee_Acq" dans la table "Vehicule" est entre 2000 et 2022 ALTER TABLE Vehicule ADD CONSTRAINT CHECK Annee_Acq BETWEEN 2000 AND 2022; <i>Remarque:</i> Si on veut attribuer la ville " Tunis " comme valeur par défaut à tous les clients, il ne s'agit pas d'ajouter une contrainte mais de modifier la colonne " Ville " ALTER TABLE Client MODIFY COLUMN Ville VARCHAR(20) DEFAULT 'Tunis';
	Modification de la clé primaire d'une table	Modifier la clé primaire actuelle de la table Location par la clé primaire composée suivante: ImmatVehicule, NCIN, Date Loc ALTER TABLE Location DROP PRIMARY KEY ADD PRIMARY KEY (immat_Vehicule,NCIN_Cli,date_loc);
	Suppression d'une contrainte	Supprimer la clé primaire de la table " Client " ALTER TABLE Client DROP PRIMARY KEY;
	Activer / désactiver une contrainte	Désactiver la contrainte de clé primaire de la table " client " ALTER TABLE Client DISABLE KEYS; Réactiver cette même contrainte ALTER TABLE Client ENABLE KEYS
	Renommer une table	Renommer la table " Vehicule " en " Voiture " ALTER TABLE Vehicule RENAME TO Voiture;
Suppression d'une table		Supprimer la table " Client " DROP TABLE Client;
Suppression de la base de données		Supprimer la base de données " Gestion_Location " DROP DATABASE Gestion_Location;

Partie LMD

<u>Objet</u>	<u>Méthode prédéfinie</u>	<u>Exemple d'utilisation</u>
Clauses	GROUP BY, ORDER BY, HAVING:	Select Cod_cli , count (*) "Nombre d'achat" From ventes Group by Cod_cli Having count (*)>=2;
Opérateurs	DISTINCT, BETWEEN, LIKE, IN, AND, OR, =, !=, <, >	
Fonctions d'agrégation	Count	→ <u>Nombre des articles fournis par le fournisseur numéro 777</u> Select count (*) From Articles Where num_frs=777;
	AVG	→ <u>La moyenne des prix d'achat des articles</u> Select avg (pa_art) From articles; <u>Ou bien</u> Select sum(pa_art)/count(*) From articles;
	Max	→ <u>L'article le prix plus cher (ayant le prix de vente le plus élevé)</u> Select max (pv_art) From articles; → <u>La désignation de l'article le prix plus cher (ayant le prix de vente le plus élevé)</u> Select des_art "Article plus cher" From articles Where pv_art=(select max(pv_art) From articles);
	Min	→ <u>L'article le prix moins cher (ayant le prix de vente le plus bas)</u> Select min (pv_art) "Prix bas" From articles; → <u>Le numéro de l'article le moins cher (ayant le prix de vente le plus cher)</u> Select num_art From articles Where pv_art= (select min(pv_art) From articles);
	Sum	→ <u>La somme des prix d'achat des articles</u> Select sum(pa_art) From articles;

Fonctions de chaînes de caractères	Concat : permet de concaténer les valeurs de plusieurs colonnes pour ne former qu’une seule chaîne de caractère. Cette fonction SQL peut se révéler pratique pour mettre bout-à-bout les valeurs de plusieurs colonnes pour n’en afficher qu’une	➔ <u>Concaténer le prénom et le nom</u> SELECT id, CONCAT(prenom, ' ', nom) AS affichage_nom, date_inscription FROM utilisateur; ➔ <u>Recherche sur le prénom + le nom d’un coup</u> SELECT id, prenom, nom, date_inscription FROM utilisateur WHERE CONCAT(prenom, ' ', nom) LIKE 'Mohamed Ouni';
	Length : Dans le langage SQL la fonction LENGTH() permet de calculer la longueur d’une chaîne de caractères. A noter : la fonction se base sur le nombre d’octets. Un caractère multi-octet compte comme un seul caractère. Autrement dit, un caractère tel que “œ” compte comme un seul caractère.	SELECT LENGTH('exemple'); ➔ Résultat : 7 ➔ Extraire les utilisateurs avec un numéro de téléphone inférieur à 10 caractères SELECT id, login, ville, telephone FROM utilisateur WHERE LENGTH(telephone) < 10;
	Substring : La fonction SUBSTRING() dans le langage SQL (ou SUBSTR()) est utilisée pour segmenter une chaîne de caractère. Autrement dit, cela permet d’extraire une partie d’une chaîne, par exemple pour tronquer un texte. Cette fonction peut s’utiliser de 4 façons différentes, que voici : ▪ SUBSTRING(chaîne, debut) : retourne la chaîne de caractère de “chaîne” à partir de la position définie par “debut” (position en nombre de caractères) ▪ SUBSTRING(chaîne FROM debut) : idem que précédent ▪ SUBSTRING(chaîne, debut, longueur) : retourne la chaîne de caractère “chaîne” en partant de la position définie par “debut” et sur la longueur définie par “longueur” ▪ SUBSTRING(chaîne FROM debut FOR longueur) : idem que précédent	Exclure le premier caractère SELECT SUBSTR(nom_colonne, 2) AS colonne_sans_first FROM table WHERE condition; Exclure le dernier caractère SELECT nom_colonne, SUBSTR(nom_colonne, 1, CHAR_LENGTH(nom_colonne) - 1) AS colonne_sans_last FROM table WHERE condition; <u>Voir plus d’exemples dans la partie Update</u>
	Left : permet de retourner le nombre souhaité de caractères parmi les premiers caractères d’une chaîne. Cette fonction va tronquer volontairement le texte d’entrée à la longueur désiré. A noter : c’est similaire à la fonction SUBSTR() qui permet d’extraire un bout de chaîne, à la différence qu’elle permet uniquement d’extraire les caractères situé au début.	SELECT LEFT(chaîne, longueur); SELECT LEFT('abcdefghij', 2); ➔ résultat : 'ab' SELECT LEFT('abcdefghij', 40); ➔ résultat : 'abcdefghij' SELECT LEFT('abcdefghij', 0); ➔ résultat : '' SELECT LEFT('abcdefghij', -1); ➔ résultat : '' SELECT LEFT('abcdefghij', NULL); ➔ résultat : NULL

	Right : permet d’extraire la fin d’une chaîne de caractères en définissant la longueur souhaitée. La fonction permet de tronquer une chaîne en partant de la fin, sur le nombre de caractères souhaités.		<pre>SELECT RIGHT(chaine, longueur); SELECT RIGHT('abcdefghij', 3); → résultat : 'hij' SELECT RIGHT('abcdefghij', 40); →résultat : 'abcdefghij' SELECT RIGHT('abcdefghij', 0); →résultat : '' SELECT RIGHT('abcdefghij', -1); →résultat : '' SELECT RIGHT('abcdefghij', NULL); →résultat : NULL</pre>								
Date, TIME, DATETIME, YEAR	<table><tr><td>DATE</td><td>Date (ex: 2024-11-04)</td></tr><tr><td>TIME</td><td>Heure (ex: 23:44:05)</td></tr><tr><td>DATETIME</td><td>Date et heure (ex: 2024-11-04 23:44:05)</td></tr><tr><td>YEAR</td><td>Année (ex: 2021)</td></tr></table>	DATE	Date (ex: 2024-11-04)	TIME	Heure (ex: 23:44:05)	DATETIME	Date et heure (ex: 2024-11-04 23:44:05)	YEAR	Année (ex: 2021)		<u>Exemple :</u> <pre>CREATE TABLE Location(immat_Vehicule VARCHAR(10) REFERENCES vehicule(immat_Vehicule)ON DELETE CASCADE, NCIN_Cli VARCHAR(8) REFERENCES Client(NCIN_Cli)ON DELETE CASCADE, date_loc TIMESTAMP DEFAULT CURRENT_TIMESTAMP CHECK(date_loc<=NOW()), duree_loc INT(3) CHECK(duree_loc>0), cout_loc DECIMAL(8,3) CHECK(cout_loc>0), CONSTRAINT PK_Location PRIMARY KEY(immat_Vehicule,NCIN_Cli));</pre>
	DATE	Date (ex: 2024-11-04)									
	TIME	Heure (ex: 23:44:05)									
DATETIME	Date et heure (ex: 2024-11-04 23:44:05)										
YEAR	Année (ex: 2021)										
Pour attribuer la date d'aujourd'hui comme valeur par défaut, il faut utiliser le type "timestamp" sinon ça ne marche pas avec le type date											
	Timestamp : La fonction SQL TIMESTAMP() est utilisée dans le langage SQL pour obtenir un DATETIME à partir d’une DATE. La fonction peut contenir un à deux arguments : • Avec 1 argument : une date au format DATE est transformée en DATETIME • Avec 2 arguments : une date et heure seront transformée en DATETIME		SELECT TIMESTAMP("YYYY-MM-DD"); -- résultat sous la format YYYY-MM-DD HH:MM:SS <u>Exemple :</u> Obtenir une date/heure à partir d’un champ qui ne contiendrait qu’une date. <pre>SELECT TIMESTAMP("2024-09-15"); -- résultat sous la format 2024-09-15 00:00:00</pre> SELECT TIMESTAMP("YYYY-MM-DD", "HH:MM"); -- résultat sous la format YYYY-MM-DD HH:MM:SS <u>Exemple :</u> Une date soit enregistrée dans une colonne, tandis que l’horaire est enregistré dans une autre colonne. Cette fonction serait alors utilisée pour combiner la date et heure en une seule information au format DATETIME. <pre>SELECT TIMESTAMP("2024-09-15", "10:35"); -- résultat sous la format 2024-09-15 10:35:00</pre>								

	Day: • DAY() synonyme de DAYOFMONTH() • DAYNAME() retourne le nom du jour de la semaine • DAYOFMONTH() retourner le jour dans le mois (de 1 à 31) • DAYOFWEEK() retourner le jour dans la semaine (1=dimanche, 2=lundi, 3=mardi ...) • DAYOFYEAR() retourner le jour dans l'année (de 1 à 366)	
	Month: permet d'extraire le numéro de mois à partir d'une date au format AAAA-MM-JJ.	<pre>SELECT MONTH(date); SELECT MONTH('2024-03-06'); -- résultat : SELECT MONTH('2025-01-01'); -- résultat : 1 SELECT MONTH('1999-12-01'); -- résultat : 12 SELECT MONTH('1802-07-07'); -- résultat : 7</pre> <u>Important : il est important de noter que la fonction ne retourne pas les zéros pour les mois de '01' à '09'.</u>
	Year: Dans le langage SQL la fonction YEAR() permet d'extraire une année à partir d'une date au format AAAA-MM-JJ.	<pre>SELECT YEAR(date); SELECT YEAR('2024-03-01'); -- résultat : 2024 SELECT YEAR('1999-12-31'); -- résultat : 1999 SELECT YEAR('1914-06-06'); -- résultat : 1914 SELECT YEAR('1801-12-31'); -- résultat : 1801</pre>
	Now: permet de retourner la date et l'heure du système.	SELECT NOW(); 2024-02-22 16:19:43
	La fonction DATEADD() : ajoute un intervalle heure/date à une date, puis renvoie la date. Syntaxe: DATEADD(interval, number, date)	<pre>SELECT DATEADD(year, 1, '2024/08/25') AS DateAdd; -- donne 2025-08-25 00:00:00.000 SELECT DATEADD(month, 3, '2024/08/25') AS DateAdd; -- donne 2024-11-25 00:00:00.000 SELECT DATEADD(month, -2, '2024/12/25') AS DateAdd; -- donne 2024-10-25 00:00:00.000 SELECT LastName, BirthDate, DATEADD(year, 18, BirthDate) AS DateAdd FROM Employees;</pre>
	La fonction DATEDIFF() renvoie la différence entre deux dates, sous forme d'entier. Syntaxe: DATEDIFF(interval, date1, date2)	<pre>SELECT DATEDIFF(year, '2024/08/25', '2011/08/25') AS DateDiff; -- donne -13 SELECT DATEDIFF(hour, '2024/11/25 07:00', '2024/11/25 10:45') AS DateDiff; -- donne 3</pre>

UPDATE	La commande UPDATE permet d'effectuer des modifications sur des lignes existantes. Très souvent cette commande est utilisée avec WHERE pour spécifier sur quelles lignes doivent porter la ou les modifications.	UPDATE table SET colonne_1 = 'valeur 1', colonne_2 = 'valeur 2', colonne_3 = 'valeur 3' WHERE condition ; <u>Exemple :</u> augmenter les salaires des employés qui ont dépassé 10 ans de 15% UPDATE Employé SET Salaire = Salaire * 1.15 Where anciennete > 10 ;
	Supprimer le premier caractère	UPDATE table SET nom_colonne = SUBSTR(nom_colonne, 2) WHERE condition ;
	Supprimer le dernier caractère	UPDATE table SET nom_colonne = SUBSTR(nom_colonne, 1, CHAR_LENGTH(nom_colonne) - 1) WHERE condition ;

Travail élaboré par A.Essouyah